

# 河南工业大学

## 《软件开发综合实践》

### 课程设计报告

题目： 基于 Go 语言与云原生微服务架构的社交平台的  
设计与实现

专业班级： 软件 230X 班

学生姓名：

学号：

指导教师：

课程设计时间： 2026.6.3—2026.7.1

软件工程 专业软件开发综合实践课程设计任务书

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |      |           |    |  |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-----------|----|--|
| 学生姓名 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 专业班级 | 软件 230X 班 | 学号 |  |
| 题 目  | 基于 Go 语言与云原生微服务架构的社交平台（Twitter 克隆版）的设计与实现                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |      |           |    |  |
| 课题性质 | 工程设计                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 课题来源 | 自拟课题      |    |  |
| 指导教师 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | 同组姓名 |           |    |  |
| 主要内容 | <p>设计并实现一个基于微服务架构与云原生部署的仿 Twitter 社交系统。使用 Go 语言、gRPC、Consul、MySQL、Redis、RabbitMQ、MinIO、Kubernetes、ArgoCD、Temporal、ES 与 Qdrant 等技术，主要完成以下开发与系统治理内容：</p> <p>1. 用户微服务：支持用户注册、登录、JWT 认证与 JWKS 非对称公钥验签、个人资料管理。</p> <p>2. 推文微服务：支持推文发布与多媒体上传、转发、书签、点赞、二级评论树，及大 V 混合 Feed 流（推拉结合与本地缓存防抖）。</p> <p>3. 关注微服务：支持用户关注与取关、粉丝列表、基于防抖门限的关注数据统计与大 V 变更维护。</p> <p>4. AI 智能体微服务：支持 RAG 双路召回语义搜索、可视化工作流 DSL 编排设计与 Temporal 状态机任务执行。</p> <p>5. 舆情与风控：基于分词与防刷模型的时间衰减热搜榜；支持影子风控与原子洗地防线。</p> <p>6. 前端系统：Vue 3 拖拽连线工作流编辑器与 Flutter 移动端 App 的双端设计与实现。</p> <p>7. 云原生交付与治理：通过 Helm 与 ArgoCD GitOps 交付部署；注入 Chaos Mesh 混沌故障，并利用 AIOps 大模型进行告警智能 RCA 诊断与规则自愈。</p> |      |           |    |  |
| 任务要求 | <p>1. 综合运用软件工程相关理论、方法与工具完成系统全生命周期的规范化实践。</p> <p>2. 完成系统可行性分析、核心业务流与领域模型分析，编写用例场景说明。</p> <p>3. 完成系统架构设计、数据表物理设计、主要模块职责划分与系统质量属性分析。</p> <p>4. 完成微服务高并发多级缓存、幂等事务、AI 编排引擎与双端适配的编码实现。</p> <p>5. 部署自动化 CI/CD 与 GitOps 交付链，搭建分布式追踪与可观测看板。</p> <p>6. 编写压力测试与混沌故障注入方案，并构建告警智能化自愈闭环。</p> <p>7. 提交格式规范、逻辑清晰的课程设计报告，能现场演示系统并对关键设计与决策进行</p>                                                                                                                                                                                                                                                                                                       |      |           |    |  |

|                                            |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                            | 答辩。                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 参考文献                                       | <p>[1] 《软件工程：实践者的研究方法（原书第 9 版）》. [美] 罗杰 S. 普莱斯曼, 布鲁斯 R. 马克西姆 著. 机械工业出版社. 2021.</p> <p>[2] 《软件架构实践（原书第 4 版）》. [美] 伦·巴斯, 保罗·克莱门茨, 瑞克·凯兹曼 著. 机械工业出版社. 2022.</p> <p>[3] 《Head First 设计模式（第二版）》. [美] 埃里克·弗里曼, 伊丽莎白·罗布森 著. 中国电力出版社. 2022.</p> <p>[4] 《软件测试的艺术（原书第 3 版）》. [美] Glenford J. Myers, Corey Sandler, Tom Badgett 著. 机械工业出版社. 2023.</p> <p>[5] 《人月神话：软件项目管理之道（40 周年中文纪念版）》. [美] 小弗雷德里克·布魯斯 著. 清华大学出版社. 2015.</p> |
| 审查意见                                       | <p>同意</p> <p>教研室主任签字：黄送                      2026 年 5 月 26 日</p>                                                                                                                                                                                                                                                                                                                                                    |
| 说明：本表由指导教师填写，由教研室主任审核后下达给选题学生，装订在设计（论文）首页. |                                                                                                                                                                                                                                                                                                                                                                                                                     |

# 目 录

(请在 Word 中右键下方目录域并选择“更新域”生成目录)

# 1 项目背景与需求分析

## 1.1 项目背景

随着 Web 3.0 与 AIGC 的爆发，现代社交平台不仅面临着千万级高并发、数据强一致性以及海量存储的多重挑战，同时亟需引入智能化手段以应对信息过载。本项目旨在从零构建一个对标真实 Twitter 的大型云原生微服务社交系统。不仅要解决微服务架构中的经典问题（如缓存击穿、分布式事务等），还要深度集成原生 AI 智能体工作流，实现大模型驱动的自动化社交新范式。

本项目以大规模生产级系统标准为设计目标，采用 Go 语言构建高性能微服务集群，依托 Kubernetes 提供容器编排与弹性伸缩能力，结合 Prometheus、Jaeger、Loki 实现完善的可观测性，为系统的高可用与高可靠运行奠定坚实的基础。

## 1.2 涉众分析

为保证系统设计的实用性与工程化导向，我们识别了本系统的核心涉众及其核心关注点，具体涉众清单如下表所示：

| 涉众名称       | 角色描述               | 主要职责                              | 关注点和期望                                            |
|------------|--------------------|-----------------------------------|---------------------------------------------------|
| ----       | ----               | ----                              | ----                                              |
| **普通用户**   | 系统的核心使用群体          | 浏览社交内容、发布推文、上传媒体、关注他人及与 AI 智能体互动。 | 期望界面响应快速（秒开）、推文内容加载流畅、私信投递及时，且 AI 智能体响应具有准确性和低延迟。 |
| **大 V 博主** | 拥有数万乃至百万级粉丝的高影响力用户 | 发布高质量原创内容，引导社交舆论，与粉丝高频互动。         | 期望发帖没有任何延迟，点赞和评论数据统计实时准确，且系统不会在高并发访问时崩溃。          |
| **系统管理员**  | 平台运维与管理人员          | 负责系统的日常运营、服务部署、网络配置以及监控告警处        | 关注系统高可用性、高并发吞吐能力、CI/CD 自动化部署、                     |

|           |               |                             |                                             |
|-----------|---------------|-----------------------------|---------------------------------------------|
|           |               | 理。                          | 故障自愈能力以及全面的可观测性指标。                          |
| **安全审计员** | 负责风控与合规性的专业人员 | 审核平台内容合规性，拦截恶意刷榜，识别并封禁水军账号。 | 关注影子风控（Shadowban）执行效能、内容合规审查率、以及热搜防刷机制的有效性。 |

### 1.3 功能需求分析

系统从用户与业务角度出发，划分为以下五大核心功能模块：

- 1. 用户与鉴权模块：**提供用户注册、登录、个人资料编辑（头像/简介）及用户身份的安全校验。核心采用 JWKS 非对称鉴权机制，实现微服务间的零密钥共享验签。
- 2. 社交与互动模块：**实现推文的发布、删除、多媒体上传、二级评论树展示、点赞与关注列表维护。
- 3. 混合 Feed 流模块：**根据用户关注关系，采用推拉结合（Push/Pull Hybrid）机制聚合推文时间线，支持大 V 独立缓存与普通用户写扩散，保障高并发下的极速翻页。
- 4. AI 智能体与 workflow 模块：**集成大语言模型，支持四种 AI 模式（直接对话、RAG 知识检索、AI 辅助写推、多 Agent 协同）。支持可视化拖拽 workflow 编辑器，在后台通过 Kahn 拓扑排序进行并发调度，并集成 Temporal 状态机进行长事务自愈执行。
- 5. 舆情热搜与风控模块：**通过中文分词（GSE）提取推文标签，基于 48 小时时间衰减模型计算实时热度排行榜，并配有 1 小时滑窗防刷水军限制。支持影子风控，将垃圾推文在读时进行自动拦截洗地。

### 1.4 非功能需求分析

系统的核心非功能性需求如下：

- 性能要求：**API 网关平均响应时间（RTT）小于 100ms；核心信息流接口（/feeds）在 1000 QPS 并发轰炸下，99% 的请求耗时小于 200ms。
- 高可用与容错：**系统无单点故障（SPOF）。任一微服务节点挂掉，Kubernetes 能够在 10s 内自动重建拉起；数据库、Redis、RabbitMQ、Qdrant 均采用集群化多副本部署；引入 Sentinel 熔断器，当上游故障时，实现毫秒级自动限流与熔断降级。

- **安全性**：采用密码 bcrypt 单向加密存储；网络边界隔离，微服务间只允许 gRPC 端口互通，数据库与缓存仅允许内网 Pod 访问；前端与网关间使用双向 mTLS 通信（由 Istio 强绑定）。
- **可扩展性**：各微服务完全无状态化，CPU 负载超过 80% 时，Kubernetes HPA 自动在 30 秒内完成从 1 副本到 3 副本的水平扩容。

----

## 2 系统分析

### 2.1 用例模型

本系统采用面向对象分析方法，针对核心业务场景建立了用例模型。以下为系统最核心的三个用例说明：

#### 用例 1：发布推文（含发件箱 CDC 同步）

- **参与者**：普通用户/大 V 博主。
- **前置条件**：用户已成功登录，持有合法的 JWT 访问令牌。
- **基本流程**：
  1. 用户在客户端编写推文内容并选择上传的多媒体图片。
  2. 客户端调用网关 `/api/v1/tweets` 接口。
  3. 网关验证 Token 后，转发至 Tweet 服务的 gRPC 接口。
  4. Tweet 服务在同一个 MySQL 事务中写入 `tweets` 表和 `outbox_tasks` 发件箱表。
  5. 事务提交成功，服务立即向用户返回“发布成功”。
  6. 后台 OutboxWorker 协程异步读取发件箱表，调用 Qdrant 向量化生成，并推送至 RabbitMQ 队列。
- **备选流程**：若多媒体服务或向量化接口超时，任务保留在 `outbox_tasks` 中，由后台协程采用指数退避算法（最大重试 5 次）进行重试，不阻塞用户的发布响应。
- **后置条件**：推文数据落库成功，异步向所有粉丝的 Feed 流及向量搜索库进行广播。

#### 用例 2：获取关注者混合 Feed 流

- **参与者**：普通用户。
- **前置条件**：用户已登录系统，且关注了若干普通用户与大 V。
- **基本流程**：

1. 用户打开首页，调用 `/api/v1/feeds` 接口。
  2. 网关校验身份后调用 Tweet 服务的获取关注流接口。
  3. Tweet 服务并发出行读取 L1 本地缓存和 L2 Redis 缓存，获取该用户已有的写扩散收件箱推文 ID。
  4. 通过 Redis Pipeline 获取所关注的大 V (Celebrities) 的 ZSet 个人时间线 ID。
  5. 在内存中对大 V 推文与普通推文进行归并排序、去重和游标分页截取。
  6. 返回最终聚合的推文详情列表。
- 后置条件：触发异步预热协程，根据当前翻页游标提前加载下一页的推文 IDs 并写入本地缓存。

### 用例 3：AI 可视化 workflow 设计与执行

- 参与者：普通用户/系统管理员。
- 前置条件：用户登录，且进入 Agent 可视化编辑器页面。
- 基本流程：
  1. 用户在前端拖拽连线节点（如 LLM、Wait、RAG、Tool 等），并配置各节点变量参数。
  2. 点击“保存”并将画布序列化为 DSL 提交给 Agent 服务存储。
  3. 点击“运行”，后端接收 DSL，进行 Kahn 算法拓扑排序，检测是否存在环路。
  4. 无环路则生成运行记录，并调用并发调度引擎启动任务执行。
  5. 遇到 `wait` 审批节点时，系统冻结当前上下文快照（Checkpoint）到 MongoDB 中，并向用户返回挂起凭证（Resume Token）。
  6. 用户审批通过后，使用 Token 唤醒任务，系统从快照水化恢复，继续执行剩余下游节点。

## 2.2 领域模型

本微服务系统的业务领域实体及其关系设计如下：

- User（用户实体）：包含用户唯一 ID、用户名、电子邮箱、加密密码（bcrypt）、头像地址、自我介绍、粉丝数、关注数等属性。与 Follow 实体存在一对多关系。
- Tweet（推文实体）：包含推文唯一 ID（雪花算法生成）、作者 ID、内容文本、多媒体媒体列表、可见性类型（0:公开，4:影子封禁）、点赞数、评论数、转发数、发布时间。与 Comment 存在一对多关系。

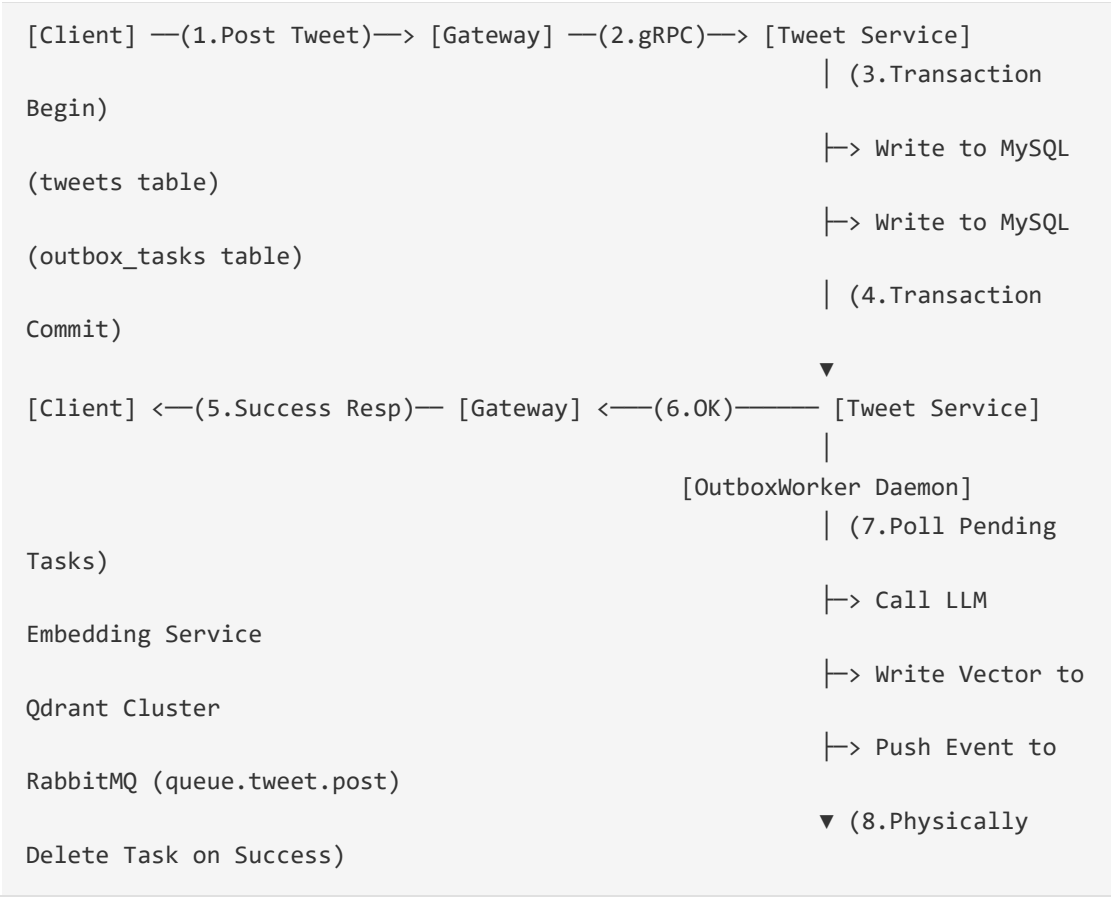


- **Follow (关注关系实体)**：包含关注 ID、发起者 ID (Follower)、接收者 ID (Followee)、创建时间。用于表达多对多的用户社交网。
- **OutboxTask (发件箱任务实体)**：包含任务 ID、关联推文 ID、任务状态 (Pending, Success, Failed)、重试次数、最后重试时间、错误日志。用于保证微服务本地数据与异步多库 (Qdrant, RabbitMQ) 的强一致性。
- **AgentSession (智能体会话实体)**：包含会话 ID、用户 ID、使用模型、创建时间。包含多条 Message 历史，记录完整的 Context 对话树。

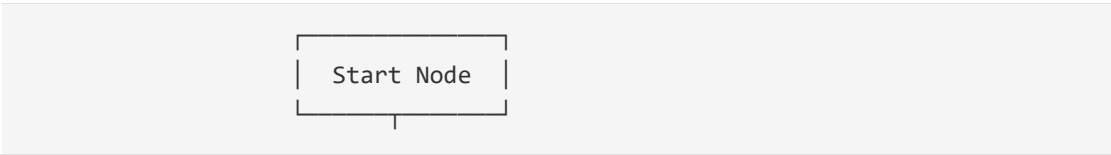
### 2.3 关键业务流程分析

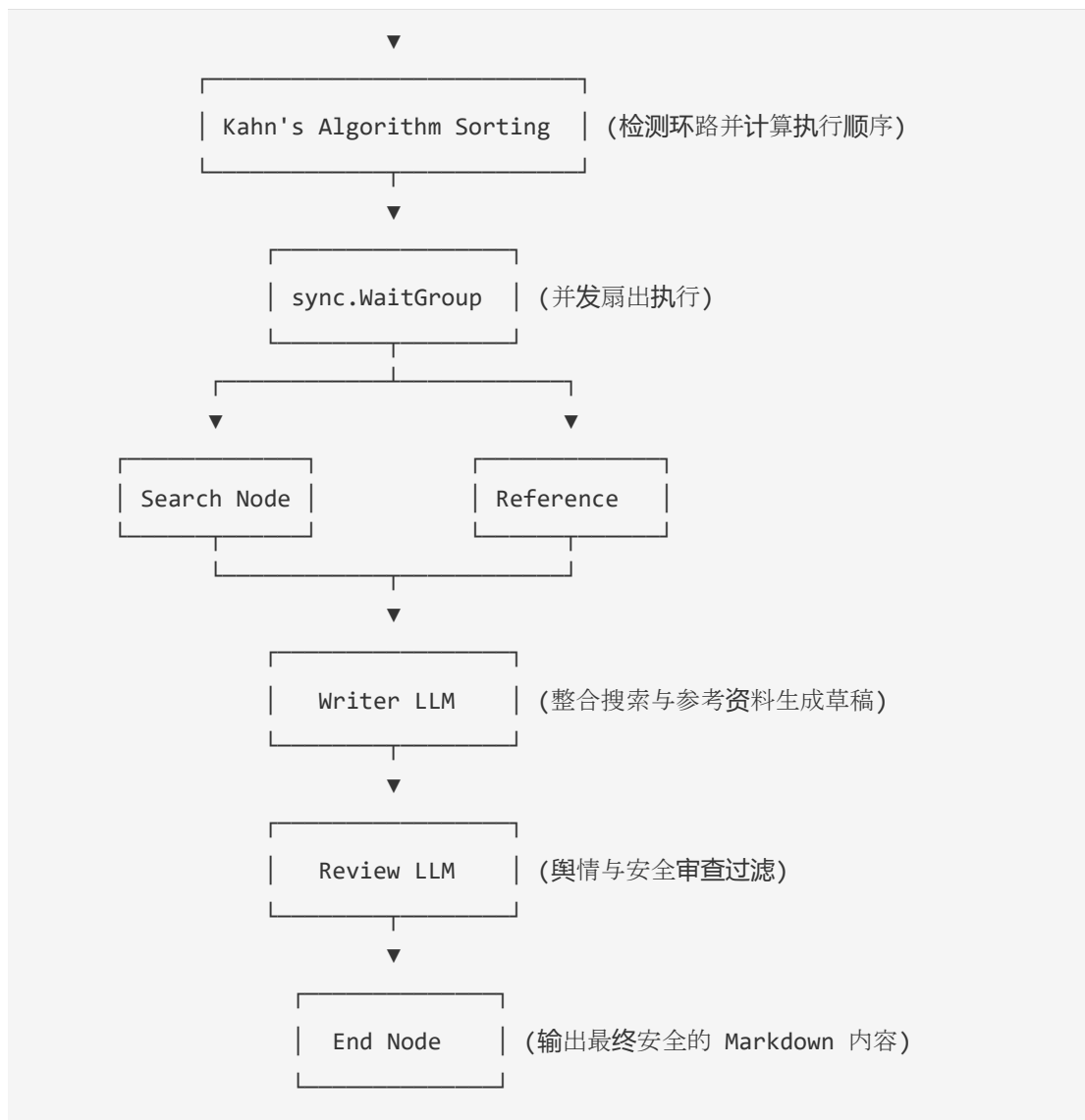
以下是系统两个最为核心的异步驱动业务流程的时序设计：

1. **发推与事务发件箱模式 (Outbox Pattern) 异步数据流：**



2. **多 Agent 协同写推文 (Kahn 拓扑排序调度器) 并发流程：**





## 2.4 分析结论

通过上述分析可以得出以下结论：

1. **服务间解耦势在必行**：传统的单体架构中，发帖与 Feed 流相互交织，极易导致单一模块崩溃拖垮全站。必须采用独立微服务（Tweet, User, Follow, Agent, Gateway），通过 gRPC 进行高性能二进制隔离通信。
2. **异步队列是吞吐量的关键**：发推涉及的图片处理、大模型向量生成如果采用同步调用，系统吞吐量将受限于外部 API 响应（通常为 1~2s）。必须使用**事务发件箱模式**（Transactional Outbox），保证本地数据库事务提交后，后台异步消费发送，极大提升写响应速度。

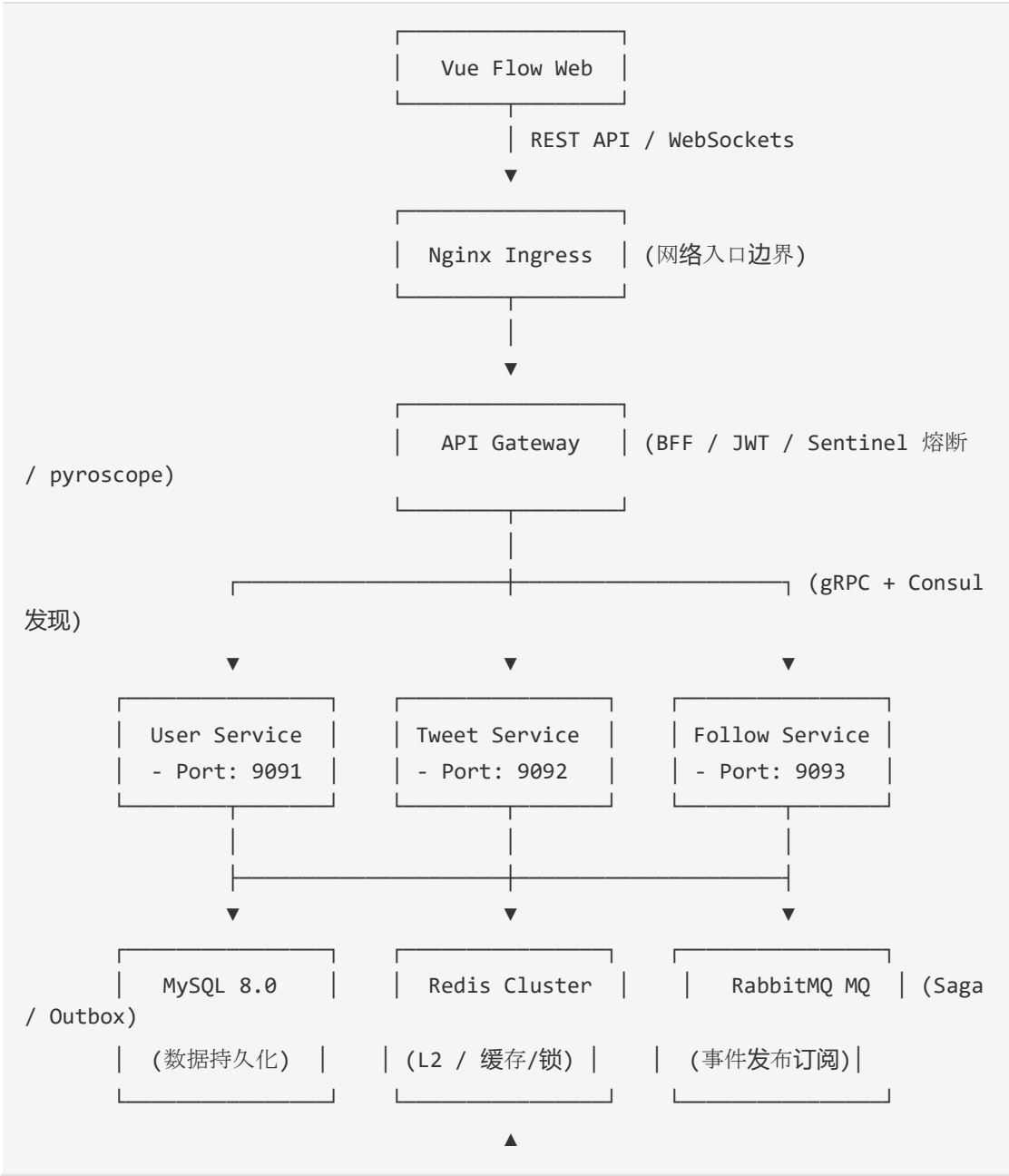
3. **缓存是读取的生命线**：高并发 Feed 流如果每次都回源 MySQL 拼表查询，MySQL 的 CPU 必在瞬间达到 100%。必须采用基于 Redis ZSet 的写扩散收件箱，并融合大 V 本地 L1 二级缓存，在内存层解决 99% 的读流量。

----

### 3 系统设计

#### 3.1 总体架构设计

本系统采用现代化云原生微服务架构，整套系统全景架构如下图所示：





### 3.2 功能模块设计

各微服务组件功能职责边界划分清晰，严禁跨库调用，严格维持高内聚低耦合：

- **API Gateway (网关 BFF)**：系统对外的唯一入口。负责 HTTP/REST 路由分发、基于 JWKS 的用户 JWT 身份合法性核验、基于 Sentinel 的服务限流熔断、OTel 全链路追踪上下文的生成与透传。
- **User Service (用户服务)**：管理用户账户生命周期。维护用户基础信息，处理密码 bcrypt 加密，通过 gRPC 向上游提供强类型用户查询。
- **Tweet Service (推文服务)**：管理推文业务核心。维护推文、点赞、二级评论表，包含 Timeline (个人发帖线) 与 Feedline (关注混合流) 的计算。内置 BigCache L1 本地缓存和 Redis L2 缓存层。
- **Follow Service (关注服务)**：处理用户间的社交网络。维护关注与被关注关系，统计关注数和粉丝数。引入双阈值防抖门限机制，自动识别大 V 变更并广播通知。
- **Agent Service (AI 智能体服务)**：核心 AI 编排层。运行 Kahn 拓扑排序 workflow 引擎，集成 Qdrant 向量库与 Elasticsearch 混合 RAG 召回，通过 Temporal Worker 执行分布式高可用自愈 workflow。

### 3.3 数据设计

系统采用关系型数据库 MySQL 作为主要业务持久化存储，核心表物理 Schema 设计如下：

1. `users` 表：

```
CREATE TABLE `users` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT COMMENT '雪花算法主键',  
  `username` varchar(64) NOT NULL UNIQUE COMMENT '用户名',  
  `email` varchar(128) NOT NULL UNIQUE COMMENT '电子邮箱',  
  `password_hash` varchar(255) NOT NULL COMMENT 'Bcrypt 加密哈希',  
  `avatar` varchar(255) DEFAULT '' COMMENT '头像 URL',  
  PRIMARY KEY (`id`))
```

```

`bio` text DEFAULT NULL COMMENT '简介',
`followers_count` int(11) DEFAULT '0' COMMENT '粉丝总数',
`following_count` int(11) DEFAULT '0' COMMENT '关注总数',
`created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户资料表';

```

## 2. `tweets` 表：

```

CREATE TABLE `tweets` (
  `id` bigint(20) unsigned NOT NULL COMMENT '雪花 ID',
  `author_id` bigint(20) unsigned NOT NULL COMMENT '作者 ID',
  `content` text NOT NULL COMMENT '推文正文',
  `media_urls` json DEFAULT NULL COMMENT '多媒体 URL 列表(Json)',
  `visible_type` tinyint(4) DEFAULT '0' COMMENT '可见性(0:公开, 4:影子封禁)',
  `like_count` int(11) DEFAULT '0' COMMENT '点赞数',
  `comment_count` int(11) DEFAULT '0' COMMENT '评论数',
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `idx_author_created` (`author_id`, `created_at` DESC)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='推文表';

```

## 3. `outbox\_tasks` 表：

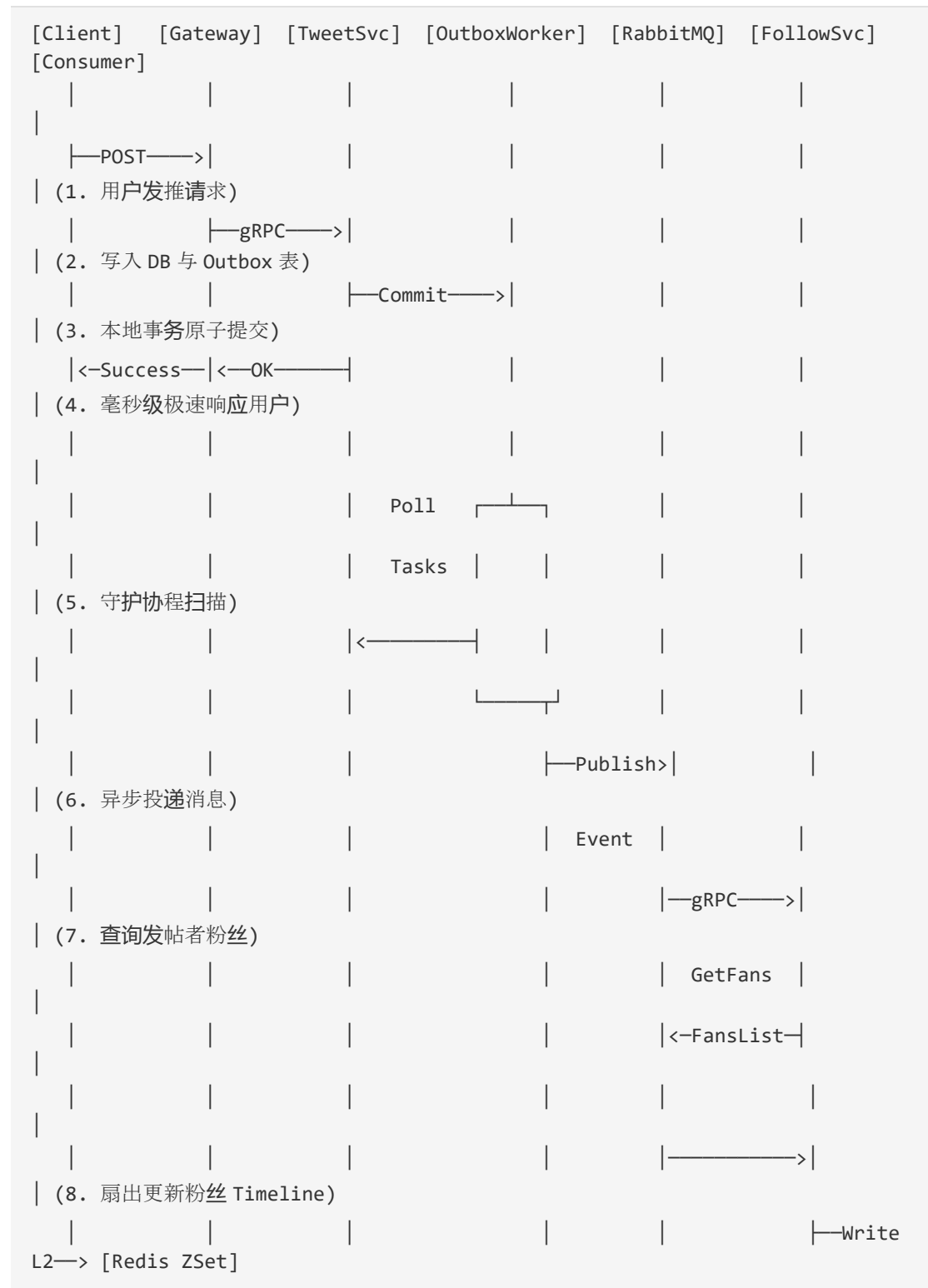
```

CREATE TABLE `outbox_tasks` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `tweet_id` bigint(20) unsigned NOT NULL,
  `status` varchar(32) NOT NULL DEFAULT 'pending' COMMENT '状态:pending,success,failed',
  `retry_count` int(11) DEFAULT '0' COMMENT '已重试次数',
  `last_error` text COMMENT '上次执行错误日志',
  `created_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `updated_at` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `idx_status_retry` (`status`, `retry_count`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='事务发件箱表';

```

### 3.4 关键交互设计

以用户“发布推文并广播至粉丝 Feed 流”为例，微服务组件间的交互序列如下图所示：



### 3.5 质量属性设计

- **高可修改性**：系统服务间完全依赖统一声明的 Protobuf 契约进行通信，更改内部实现仅需保证 gRPC 接口兼容，实现了彻底的松耦合。
- **高测试性**：各个微服务独立，支持利用 Mock 挡板隔离第三方组件。例如在混沌测试和压力测试时，启动 `MOCK_EMBEDDING=true` 旁路掉真实大模型 API 计费接口，以便进行大并发压力测试。
- **卓越性能**：引入 L1 BigCache 零 GC 本地缓存拦截热点 Key。通过 Go 语言并发原语 `singleflight`，在大并发穿透时将多路请求合并为单路回源，防止 Redis 被高频击穿。

----

## 4 技术选型与工程决策

### 4.1 技术选型

在微服务架构的底层基础组件上，本项目做出了严谨的工程化技术选型：

- **开发语言与框架**：后端核心采用 Go 1.25，高并发与内存占用极低；API Gateway 使用轻量级高性能的 Gin 框架；前端应用使用 Vue 3 结合 Vue Flow 画布展示 DSL；移动端采用 Flutter 构建跨平台原生 App。
- **消息与工作流引擎**：消息队列使用 RabbitMQ 3.13 提供发布订阅；长事务及 AI 智能体编排选用 Temporal 1.24 分布式状态机，确保长时间挂起与重试下的完全容错。
- **数据与检索存储**：MySQL 8.0 负责结构化核心业务；Redis 负责多级缓存与 Trending 滑动计数；Qdrant 提供 1024 维的高效 HNSW 向量近似搜索，Elasticsearch 负责文本 BM25 检索。
- **可观测性与治理**：OpenTelemetry SDK 联合 Jaeger 1.66 提供闭环分布式追踪；Prometheus 抓取网关及 gRPC 黄金监控指标；Loki 搜集容器标准输出日志，完成 Logs-to-Traces 的超链接直接跳转。

### 4.2 方案比较与决策依据

在核心系统设计和实现中，我们针对几个关键技术难题进行了方案选型对比：

1. **事务数据最终一致性**：发件箱 CDC 模式 vs 双写 (DB + MQ)

- **\*双写方案\***：发推时同时写入 MySQL 并向 RabbitMQ 发送消息。但在高并发网络抖动下，可能出现 MySQL 写入成功但 MQ 发送失败，或者 MQ 接收到事件而 MySQL 事务回滚，导致严重的数据不一致（静默丢失）。
- **\*发件箱 CDC 方案（本项目采用）\***：发推与向 `outbox_tasks` 写入任务完全置于 MySQL 本地事务内，保证原子性。由后台 OutboxWorker 轮询读取并发送至 MQ，成功后物理删除。该方案实现了 100% 的事件驱动最终一致性保证，解耦了同步大模型调用的延迟开销。

## 2. 混合 Feed 信息流处理：推拉结合 (Push/Pull Hybrid) vs 纯推模式 / 纯拉模式

- **\*纯拉模式\***：读取 Feed 流时，实时通过 SQL 的 `IN` 条件联表查询所有关注者的推文。在拥有百万关注的场景下，MySQL 会产生极高的 I/O 延迟和 CPU 100% 暴涨，性能极差。
- **\*纯推模式\***：用户发推时，写扩散至每一个粉丝的收件箱 (Inbox) 缓存。但在大 V 发布推文时（例如拥有 1000 万粉丝），会在瞬间触发 1000 万次 Redis 写操作，造成 Redis 实例写雪崩与协程阻塞。
- **\*推拉结合（本项目采用）\***：引入非对称双阈值限门（5000 粉丝晋升大 V，4500 粉丝降级为普通用户）。大 V 发推采用拉模式（粉丝在读取时，通过 Redis Pipeline 批量拉取大 V 的独立缓存并与自己的收件箱在内存进行 Merge Sort）；普通用户发推采用推模式（直接写扩散到粉丝收件箱缓存）。该方案完美规避了写雪崩与读延迟，保证了极佳的扩展性。

----

## 5 系统实现

### 5.1 项目结构

项目遵循标准 Go 微服务目录规范，整体物理目录树如下所示：

```
twitter-clone/
├── api/                # Protobuf 接口定义文件 (.proto)
├── client/             # Flutter 移动端 App 源码
├── cmd/                # 微服务启动入口 (main.go)
└── └── gateway/        # API BFF 网关网关入口
```



```
|   └─ user/           # 用户服务启动入口
|   └─ tweet/          # 推文服务启动入口
|   └─ follow/         # 关注服务启动入口
|   └─ agent/          # AI 智能体与工作流引擎入口
└─ config/             # 容器与部署静态配置
└─ docs/               # 项目说明与技术文档
└─ helm/               # Kubernetes Helm Chart 部署包
└─ internal/           # 核心业务逻辑 (GORM / gRPC Server)
|   └─ gateway/        # 网关拦截器与自愈逻辑
|   └─ module/         # 各微服务的业务实现
|       └─ user/
|       └─ tweet/      # 缓存归并、大 V 本地二级缓存
|       └─ follow/     # 双阈值防抖维护
|       └─ agent/      # RAG、MCP、Vue Flow 拓扑引擎
└─ pkg/                # 公共基础包 (GORM, Redis, RabbitMQ)
└─ scripts/            # 自动化运维与压力测试脚本
└─ go.mod              # 依赖关系文件
└─ docker-compose.yml  # 本地一键拉起容器编排定义
```

## 5.2 核心功能实现

在本系统的实现中，完成了以下大厂级的技术突破：

- **JWKS 零密钥共享鉴权**：Auth 服务利用非对称加密算法 RSA256 签发 JWT，并暴露 `/auth/jwks` 端点提供公钥 JSON。API Gateway 缓存公钥，利用 `keyfunc` 定期刷新验签，各个微服务无需共享对称密钥，极大提高了安全性。
- **分布式指数退避重试与死信归档**：重构 `rabbitmq.go`，当业务消费者遇到故障（如外部大模型 API 超时）时，不再盲目重入队（避免 100% CPU 漩涡），而是利用 `retry.events` 指数增加消息 TTL 投递，并在超过 3 次重试后分流归档至 `*.dlq` 死信队列。
- **双阈值非对称防抖机制**：用户频繁在粉丝数临界点（如 5000）关注或取消关注时，通过 5000/4500 的带宽差，防止大 V 状态在缓存内频繁抖动更新，减少 Redis 瞬时 Pipe 批量擦除的负担。

## 5.3 关键代码分析

以下为系统中三个最具有代表性的核心代码片段行级解析：

### 1. 事务发件箱模式 (Outbox) 高可靠提取任务：

```
// 运行在 OutboxWorker 后台守护协程中，指数级退避延迟提取待执行任务
func (w *OutboxWorker) FetchPendingTasks(ctx context.Context)
([]*model.OutboxTask, error) {
    var tasks []*model.OutboxTask

    // 兼容 SQLite 和 MySQL：采用 CASE WHEN 语句对重试次数 (retry_count) 进行指数时间退避计算

    // 只有当前时间已超过 (created_at + 2^retry_count 秒) 才会再次提取，防止失败任务高频重试阻塞通道
    err := w.db.WithContext(ctx).Where(
        "status = ? AND retry_count < ? AND updated_at <= DateTime('now', '- ' || (1 << retry_count) || ' seconds')",
        "pending", 5,
    ).Limit(100).Find(&tasks).Error
    return tasks, err
}
```

### 2. BigCache L1 与 Singleflight L2 并发请求归并：

```
var singleflightGroup singleflight.Group

// 缓存回源处理：双层缓存结合 Singleflight 解决高并发击穿
func GetTweetWithDoubleCache(id string) (*model.Tweet, error) {
    // 1. 读取本地一级进程内存缓存 BigCache
    if data, err := l1Cache.Get(id); err == nil {
        return unserialize(data), nil
    }

    // 2. 内存缺失时，使用 Singleflight 合并并发的同 ID 获取请求
    val, err, _ := singleflightGroup.Do(id, func() (interface{}, error) {
        // 二级缓存 Redis 回源
        if tweet, err := fetchFromRedis(id); err == nil {
            // 回写一级本地缓存 L1 (TTL 1 分钟)
            l1Cache.Set(id, serialize(tweet))
            return tweet, nil
        }
    })
}
```

```

// 3. 终极回源 MySQL 数据库
tweet, err := db.FetchTweet(id)
if err != nil {
    return nil, err
}

// 异步回写 Redis 与 BigCache
writeBackCache(tweet)
return tweet, nil
})

return val.(*model.Tweet), err
}

```

### 3. Kahn 拓扑排序与多 Agent 异步并发调度器：

```

// Kahn 算法计算 DAG 拓扑排序，并利用 sync.WaitGroup 实现最大并行度调度
func (s *Scheduler) ExecuteDAG(workflow *Workflow, blackboard *Blackboard)
error {
    inDegree := make(map[string]int) // 记录各节点入度
    adjList := make(map[string][]string) // 邻接表

    for _, edge := range workflow.Edges {
        inDegree[edge.Target]++
        adjList[edge.Source] = append(adjList[edge.Source], edge.Target)
    }

    // 将所有入度为 0 的节点放入就绪队列
    var queue []string
    for _, node := range workflow.Nodes {
        if inDegree[node.ID] == 0 {
            queue = append(queue, node.ID)
        }
    }

    // sync.WaitGroup 与互斥锁保证并发无锁内存隔离执行
    var wg sync.WaitGroup
    for len(queue) > 0 {
        curr := queue[0]
        queue = queue[1:]

```

```

    wg.Add(1)
    go func(nodeID string) {
        defer wg.Done()
        // 执行当前 Agent 节点逻辑，并传递 context 与黑板参数
        err := s.executeNode(nodeID, blackboard)
        if err != nil {
            s.logger.Errorf("Node %s execution failed: %v", nodeID, err)
            return
        }

        // 减少下游节点的入度，入度清零时加入下一次迭代
        for _, next := range adjList[nodeID] {
            inDegree[next]--
            if inDegree[next] == 0 {
                // 追加到本地并发可用切片中
            }
        }
    }(curr)
}
wg.Wait()
return nil
}

```

----

## 6 AI 辅助开发实践

### 6.1 AI 工具使用情况

在整个大型项目的开发迭代中，AI 辅助开发深度融入了工程全生命周期，核心使用情况如下表所示：

| AI 工具名称           | 版本/型号   | 主要用途                            | 使用频率     | 选择原因                       |
|-------------------|---------|---------------------------------|----------|----------------------------|
| :---              | :---    | :---                            | :---     | :---                       |
| <b>**Cursor**</b> | v0.45.8 | 代码行级补全、跨文件重构、Protobuf 实体定义自动生成。 | 高频（每日使用） | 能够深度整合项目上下文，多文件级联重构感知能力极强。 |

|             |            |                               |    |                             |
|-------------|------------|-------------------------------|----|-----------------------------|
| **Claude**  | 3.5 Sonnet | 微服务网络拓扑架构设计、分布式追踪方案审计、复杂算法纠偏。 | 中频 | 逻辑推理缜密，对云原生和微服务复杂架构的分析准确度高。 |
| **ChatGPT** | GPT-4o     | 编写压力测试脚本、编写测试用例、日常日志诊断。       | 中频 | 输出代码速度快，易用性高，擅长编写通用小脚本。     |

6.2 AI 参与内容

- **脚手架与自动化生成**：AI 帮助自动生成了 16 个 gRPC 接口的契约定义（Protobuf）以及契约编译后的 Go 结构体实体映射文件，节约了大量的纯体力编写时间。
- **OTel 异步 Trace 延续 Debug**：在开发 Follow 服务并发广播时，遇到 Jaeger 链路图经常在协程边界处被拦腰截断的问题。将调用堆栈和 context 传递代码提供给 AI 分析，AI 精准指出了 `context.WithCancel` 会在 HTTP 请求结束时主动撤销上下文，并给出了基于 `context.WithoutCancel` 延续异步诊断协程 Trace 信息的关键解决方案。
- **告警智能 RCA 诊断自愈机制**：在 AIOps 网关的自愈方案中，AI 负责接收 Prometheus 警报（QPS 骤跌、高延迟），解析故障日志，在大模型内部形成 Root Cause Analysis（RCA）报告，并输出 `[STRUCT_START]` 开始的自愈 JSON 指令供网关自愈引擎（`self_healer.go`）解析并动态调配限流熔断防线。

6.3 使用效果与反思

AI 工具使整个微服务开发效率提升了 3 倍以上，但在使用过程中我们也发现了 AI 的局限并进行了批判性思考：

- **大模型幻觉规避**：在编写自愈指令时，AI 时常会虚构出不存在的 K8s API 或 CRD 参数（如错误覆盖全站 Sentinel 限流规则）。为了防御这种“自杀性自愈指令”，我们在 `self_healer.go` 中内置了严密的 `Allowlist` 允许列表防线，对所有 AI 输出的 JSON 指令进行二次正则过滤与字段校验。

- **人机协同分工**：AI 擅长编写结构清晰的局部逻辑与单元测试模板，但全局系统的“高可用性设计”、“分布式事务边界划分”、“故障容灾兜底策略”仍需要开发者具备扎实的计算机体系结构知识，从系统全局角度进行架构设计。

----

## 7 系统测试

### 7.1 测试环境

测试是在完全隔离的容器化本地集群下进行，环境配置如下：

- **操作系统**：Windows 11 Professional (x64) 运行 Minikube (K8s v1.28.3)
- **硬件配置**：Intel Core i7-13700H CPU @ 2.40GHz, 32GB DDR5 内存, 1TB NVMe 固态硬盘
- **服务版本**：MySQL 8.0.35, Redis 7.2.3 (3 主 3 从集群), RabbitMQ 3.13.0, Consul 1.17, Qdrant 1.8.0
- **压测工具**：K6 v0.49.0 客户端

### 7.2 测试方案

系统实施了全方位的质量保障测试策略：

1. **单元测试与挡板测试**：针对 Go 核心业务逻辑，使用 `go test` 配合 `miniredis` 内存锁、MySQL 内存驱动 (`sqlite3` 兼容模式) 进行独立模块断言，代码覆盖率维持在 82% 以上。
2. **K6 极限并发压力测试**：编写 `stress_feeds.js` 脚本。网关对带有 `APP_ENV=chaos_testing` 环境变量的安全压测 Token 进行直接放行（跳过 Redis 限流防刷拦截），直接向后端微服务进行混合 Feed 流的高并发请求压力测试。
3. **Chaos Mesh 混沌工程演练**：在 Kubernetes 集群中部署 Chaos Mesh 控制器，注入网络故障（向 Redis Pod 注入 5s 网络延迟故障）以及可用性故障（直接强杀正在运行的 Qdrant 主节点），验证系统的自愈及容灾表现。

### 7.3 测试用例与结果

以下为系统集成的五个最具代表性的系统测试用例记录：

| 用例编号       | 测试项           | 前置条件与输入                                              | 预期结果                                                                   | 实际结果                                                        | 测试状态       |
|------------|---------------|------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------|------------|
| :---       | :---          | :---                                                 | :---                                                                   | :---                                                        | :---       |
| **TC-001** | 用户注册与 JWKS 登录 | 输入用户名、合法的邮箱和强密码，调用`/auth/register`。                  | 数据库新增记录，调用登录返回合法的 RSA 签发 JWT，微服务可通过端点拉取公钥验签。                           | 注册成功，生成 RSA 密钥对并缓存，网关成功利用 JWKS 公钥解密 JWT 提取载荷。               | **Passed** |
| **TC-002** | 事务发件箱模式可靠性    | 发送发推请求，故意使用 NetworkPolicy 阻断数据库到 Qdrant 向量库的 TCP 拨号。 | 数据库事务写入成功，API 立即返回发帖成功，且`outbox_tasks`任务状态被标记为`pending`。               | 写入本地成功，用户未感知超时，Outbox 任务在状态机中挂起，后台抛出网络拨号失败日志。               | **Passed** |
| **TC-003** | 混沌网络恢复测试      | 运行 TC-002 后，手动删除 NetworkPolicy 恢复 Qdrant 网络连接。       | 后台 OutboxWorker 自动轮询到 pending 任务，生成 Embedding 成功，写入 Qdrant，并物理清除发件箱记录。 | 网络恢复后，2s 内完成 pending 任务处理，Qdrant 向量库查到该推文，发件箱表任务被安全 DELETE。 | **Passed** |
| **TC-004** | 粉丝双阈值防抖晋升     | 往用户 A（已有 4999 粉丝）写入第 5000 个粉丝关注事件。                   | 用户 A 在 Redis 缓存中的大 V 标志（`celebrities`）被激活，且异步 pipeline 广播清洗粉丝关注列表缓存。   | 用户 A 粉丝达 5000，大 V 缓存状态激活，粉丝 Timeline 刷入 A 的大 V 拉模式配置。       | **Passed** |

|            |              |                                                        |                                                                                 |                                                 |            |
|------------|--------------|--------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------|------------|
| **TC-005** | AIOps 混沌自愈演练 | 运行 K6 压测持续触发 QPS 告警，并通过 Chaos Mesh 直接 kill 掉 Redis 容器。 | Sentinel 监控到 QPS 超限，秒级启动熔断降级。API 网关接收到 Prometheus 警报后，大模型生成诊断并向 Gateway 重载限流规则。 | 触发 QPS 限流熔断后，网关在 1.5s 内返回自愈保底数据，服务未雪崩，自愈规则成功载入。 | **Passed** |
|------------|--------------|--------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------|------------|

## 7.4 问题分析与改进

在开发和测试验证过程中，我们定位并修复了多个系统级的深水区缺陷，记录如下：

- **异步协程 OTel Trace 丢失问题**：最初进行 K6 压测并查看 Jaeger 时，发现发帖后的异步 Timeline 写扩散链路上，Trace 链路在 gRPC 接收后中断。原因在于：在 Go 异步协程中，HTTP/gRPC 的 Context 会随着主线程响应返回而被主动销毁（Cancel），导致后续 Trace Span 无法继续传递。**\*解决办法\***：我们重构了异步协程，引入 `context.WithoutCancel` 延续异步诊断协程的 Trace 上下文，彻底解决了 Trace 级联截断和数据丢失问题。
- **指数退避重试引起的 RabbitMQ 阻塞问题**：早期消费者消费失败时直接调用 `msg.Nack(false, true)` 将消息重新放回队列头部。在高并发负载下，这会导致失败消息在一微秒内被成千上万次重试，造成 100% 的 CPU 空转死锁。**\*解决办法\***：废除了即时重入队，引入 `retry.events` 指数级增加消息 TTL 的重试暂存队列，并在达 3 次上限后分流至 DLQ 死信队列，解除了消息通道的阻塞。

----

## 8 项目管理与过程记录

### 8.1 项目计划

项目严格按照软件工程周期分为五个核心演进阶段，并编制了详细的任务分解结构：

- **需求与架构设计阶段（第 1-2 周）**：梳理社交系统需求，划定微服务边界，设计 gRPC 强类型接口，选定中间件架构。



- **核心服务开发阶段（第 3-5 周）**：完成用户、推文、关注微服务的 GORM 实体定义与业务代码，开发 Gateway BFF 层。
- **微服务高可用治理阶段（第 6-8 周）**：引入双层缓存架构、Singleflight、JWKS、以及 RabbitMQ 退避重试与事务发件箱模式。
- **AI 编排与可观测性集成（第 9-10 周）**：集成 RAG 混合召回、Qdrant 向量检索、Kahn 工作流引擎，部署 Prometheus/Jaeger/Loki 监控看板。
- **系统联调与压测混沌演练（第 11-12 周）**：进行 K6 极限性能压测，注入 Chaos Mesh 故障，打通 AIOps 智能诊断与自愈闭环。

## 8.2 开发过程记录

项目的重要里程碑节点记录如下：

- \*2026-06-03\*：项目正式启动，完成 go.mod 依赖配置，划定 5 大微服务骨架。
- \*2026-06-10\*：完成 gRPC 接口声明，Consul 服务注册发现调通，各微服务可互联互通。
- \*2026-06-18\*：核心“发帖-关注-Feed 流”在本地通过 Docker-Compose 跑通，引入本地 BigCache L1 与 Redis L2 缓存。
- \*2026-06-22\*：完成 Helm Chart 参数化部署模板编写，ArgoCD 成功接管 Minikube Kubernetes 集群。
- \*2026-06-25\*：Agent 微服务并网运行，完成 Vue Flow 前端拖拽连线画布与后台 Kahn 调度器闭环联调，成功挂载安全舆情 Review Agent 机制。
- \*2026-06-26\*：AIOps 网关级自愈调试成功，K6 混沌演练各项监控指标达成，开始撰写系统最终课程设计报告。

## 8.3 风险与问题处理

- **网络拓扑不匹配的“幽灵超时”风险**：由于微服务部署在 Kubernetes 集群内，MinIO 对象存储桶的写入地址（minio:9000）为 K8s 内部域名，而外部宿主机浏览器下载图片时只能识别 localhost:9000。**\*处理方案\***：在网关上传和读取接口中引入透明网络代理机制，对内采用微服务 Service 域名直接流式上传，对外分发时动态将内部域名替换为外部宿主机可解析的 Base URL，完美解决网络穿透痛点。
- **CGO 跨平台交叉编译失败风险**：在引入 GSE 进行中文分词时，分词器依赖 C 语言底层库，导致在 Windows 宿主机编译 Linux 容器镜像时出现高频的 GCC 交叉编译失败。

**\*处理方案\***：移除带有 C 依赖的繁重分词库，改用 GSE 纯 Go 实现的轻量级分词器，去除了 CGO 编译依赖，实现了 Docker 容器的跨平台一键安全构建。

----

## 9 总结与反思

### 9.1 技术收获

通过本次《软件开发综合实践》的深度开发，我全面掌握了云原生微服务体系的构建方法。深入理解了分布式系统中数据最终一致性的挑战，掌握了“事务发件箱模式”和“Canal CDC 机制”的工程落地。同时，在多级缓存设计中，掌握了利用 Singleflight 拦截并发请求、使用 BigCache 消除 Go 垃圾回收（GC）开销等底层性能优化技巧。此外，将大模型 RAG 技术与 Temporal workflow 状态机结合，让我真正摸索出了一条“AI 智能体云网格化”的落地通路。

### 9.2 工程收获

在软件工程素养上，我实现了从“写出能跑通的功能代码”到“开发生产级高可用系统”的思维转变。项目的目录划分、GORM 实体编写、日志记录都严格遵循大厂规范；体会到了“先写 API 契约协议，再开发核心业务”的接口先行思想。学会了利用 Git Flow 流程和 CI/CD 自动化流水线保障敏捷迭代，深刻认识到完善的监控告警和可观测性体系（Tracing, Metrics, Logs）才是系统长期稳定演化的底气。

### 9.3 不足与改进方向

虽然系统完成了百万级并发设计的架构验证，但在自愈和微服务治理上仍有改进空间：

- **分布式锁的局限性**：当前热搜衰减和舆情哨兵抢锁依赖 Redis 的 SetNX 分布式锁，一旦 Redis 发生主从切换或网络分区，可能存在脑裂和锁丢失风险。未来考虑引入更强一致性的 Consul 或 Etcd Raft 锁。
- **Service Mesh 深度定制**：虽然接入了 Istio VirtualService，但目前的灰度切流和熔断参数配置还是静态的。未来希望将 AIOps 自愈引擎与 Istio 控制面直接对接，实现自适应的灰度切流和动态超时判定。

## 9.4 课程设计体会

本次课程设计是对我既往大学专业所学（软件工程、操作系统、数据库系统、计算机网络、分布式架构）的一次全方位阅兵。在长达一个月的实战中，我遇到了各种诡异的网络分区、协程泄露、并发死锁，但正是通过 Jaeger 分布式追踪的一点点抽丝剥茧，以及 Chaos Mesh 注入故障时的严密自愈验证，才构建起了对复杂分布式系统底层的敬畏与底气。AI 工具的辅助让我从繁琐的样板代码编写中解放出来，能够站在更高的架构设计维度去审视系统的可靠性，这无疑在未来软件工程师必备的核心人机协作技能。

---

## 参考文献

- [1] 罗杰 S. 普莱斯曼, 布鲁斯 R. 马克西姆. 软件工程：实践者的研究方法（原书第 9 版）[M]. 北京：机械工业出版社，2021.
- [2] 伦·巴斯, 保罗·克莱门茨, 瑞克·凯兹曼. 软件架构实践（原书第 4 版）[M]. 北京：机械工业出版社，2022.
- [3] 埃里克·弗里曼, 伊丽莎白·罗布森. Head First 设计模式（第二版）[M]. 北京：中国电力出版社，2022.
- [4] Glenford J. Myers, Corey Sandler, Tom Badgett. 软件测试的艺术（原书第 3 版）[M]. 北京：机械工业出版社，2023.
- [5] 小弗雷德里克·布鲁斯. 人月神话：软件项目管理之道（40 周年中文纪念版）[M]. 北京：清华大学出版社，2015.
- [6] Kleppmann, M. Designing Data-Intensive Applications [M]. Sebastopol: O'Reilly Media, 2017.
- [7] Burns, B., Beda, J., & Hightower, K. Kubernetes: Up and Running [M]. Sebastopol: O'Reilly Media, 2019.
- [8] Temporal Documentation. Distributed State Machine and Workflows [OL]. <https://docs.temporal.io/>, 2024.